

Semaphores in Process Synchronization

When many processes run at the same time, they sometimes need to use a shared resource (e.g., file, printer, memory). If multiple processes use the resource without coordination, errors happen. This is called a race condition. To avoid this, we need synchronization mechanisms. One of the most powerful tools for synchronization is the Semaphore.

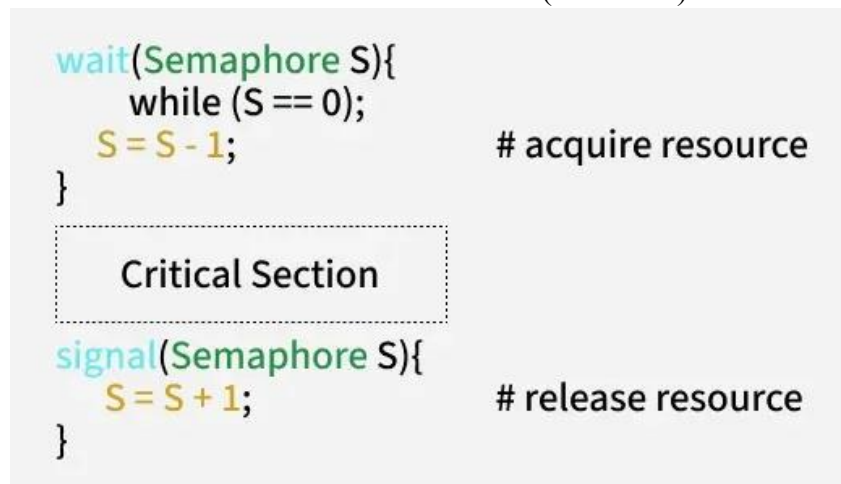
What is a Semaphore?

A Semaphore is a special variable used to control access to shared resources. It can only be changed using two atomic operations:

1. Wait () - Decrease the semaphore value. If the value became negative, the process is blocked until the resource became available.
2. Signal () → Increase the semaphore value. If there are waiting processes, one of them gets unblocked.

You can imagine a semaphore as a counter for available resources. As an example:

- Suppose a parking lot has 3 spaces.
- The semaphore value = 3.
- Each car that enters calls wait() (decrement).
- Each car that leaves calls signal() (increment).
- If the value = 0 → no more cars can enter (must wait).



Types of Semaphores

(a) Counting Semaphore

Can take any non-negative integer value (0 to N).

Used when there are multiple instances of a resource.

Example: If there are 5 printers, initial semaphore value = 5.

(b) Binary Semaphore (Mutex)

Special case of counting semaphore with only two values: 0 and 1
Works like a lock: either the resource is free (1) or busy (0).

Example: Managing access to a single critical section.

Working of Semaphore

1. Wait (S)

wait(S):

while (S <= 0)

 ; // wait (do nothing)

S = S - 1

- If $S > 0$, process continues and decreases S.
- If $S == 0$, process waits (blocked).

2. Signal (S)

signal(S):

S = S + 1

- Increases S.
- If processes are waiting, one of them is allowed to proceed.

Example: Mutual Exclusion (Critical Section)

Suppose two processes (P1 and P2) want to access a shared variable.

Semaphore S = 1; // Binary semaphore (mutex)

Process P1:

```
while (true) {  
    wait(S);    // lock  
    // critical section  
    signal(S);  // unlock  
    // remainder section  
}
```

Process P2:

```
while (true) {  
    wait(S);  
    // critical section  
    signal(S);  
    // remainder section  
}
```

}

Only **one process** can enter the critical section at a time.